MAD-2 App Development Project Report

1. Student Details

Name: Kaushik. S
Roll Number: DS23F3001827
Email: 23f3001827@ds.study.iitm.ac.in
About Me: I am a Diploma Student at IIT Madras BS Degree program with a deep interest in web application development and backend integration technologies. I enjoy building meaningful applications that combine learning, analytics, and real-world use cases.

---

2. Project Details

Project Title: Hospital Management System - V2

Problem Statement:

The Hospital Management System aims to digitize and streamline hospital operations such as patient records, appointment scheduling, and administrative tasks. It addresses inefficiencies of manual systems by providing a centralized platform for better coordination, accuracy, and real-time access to information.

Approach:

The system will be developed using a modular web-based architecture with role-based access for admins, doctors, and patients. It will implement features like authentication, appointment management, and data handling with optimized backend processing and a user-friendly frontend interface. The frontend will prioritise ease of use and a dynamic nature for different devices and screens. Backend asynchronous processes will be configured to be time-triggered or user-triggered based on use cases within a Celery server, to provide quick reminders and reports for analysis and aggregation.

---

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in writing SQLAlchemy model definitions, creating API documentation samples, and improving variable naming consistency. The extent of AI/LLM usage is around **10–15%**, limited to **code suggestions and documentation formatting**. All final implementation logic, debugging, and integration were done manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework for handling API requests and routing
SQLAlchemy	ORM for managing and interacting with the database
SQLite	Lightweight database for storing patient, doctor, and appointment data
Jinja2	Template engine for server-side rendering (if applicable)
Vue.js	Frontend framework for building dynamic and reactive user interfaces
Bootstrap 5	Responsive design and UI styling
Chart.js	Visualization of data such as patient statistics and trends
Celery	Triggering and performing backend asynchronous processes
Redis	Caching system to improve backend performance and speed

---

5. Database Schema / ER Diagram

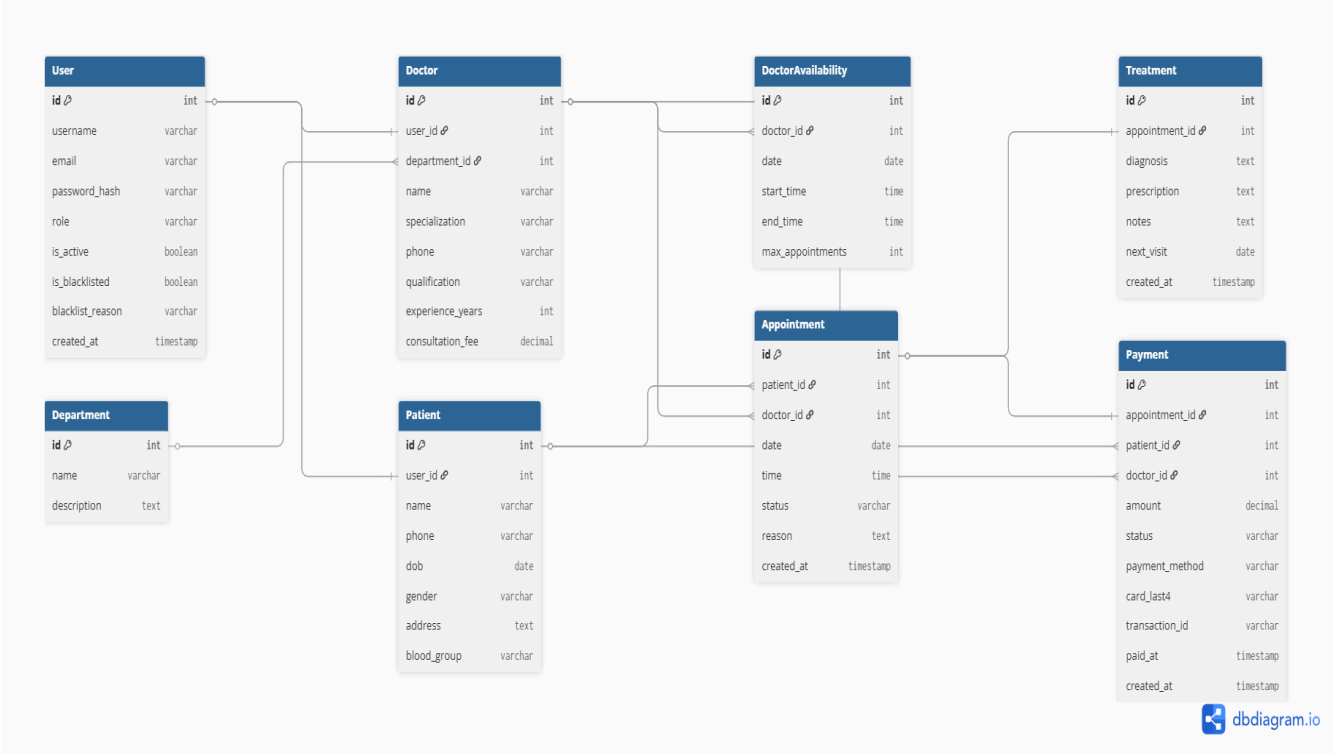
Tables:

1. **User** —stores user account details (id, username, email, password_hash, role, is_active, is_blacklisted, blacklist_reason, created_at)
2. **Department**— stores hospital department details (id, name, description)
3. **Doctor**— stores doctor profile details (id, user_id, department_id, name, specialization, phone, qualification, experience_years, consultation_fee)
4. **Patient** — stores patient profile details (id, user_id, name, phone, dob, gender, address, blood_group)
5. **DoctorAvailability**— stores doctor availability schedules (id, doctor_id, date, start_time, end_time, max_appointments)
6. **Appointment**—stores appointment booking details (id, patient_id, doctor_id, date, time, status, reason, created_at)

7. **Treatment**— stores treatment and diagnosis details (id, appointment_id, diagnosis, prescription, notes, next_visit, created_at)
8. **Payment** — stores payment transaction details (id, appointment_id, patient_id, doctor_id, amount, status, payment_method, card_last4, transaction_id, paid_at, created_at)

Relationships:

- One-to-One → User → Doctor
- One-to-One → User → Patient
- One-to-Many → Department → Doctor
- One-to-Many → Doctor → DoctorAvailability
- One-to-Many → Doctor → Appointment
- One-to-Many → Patient → Appointment
- One-to-One → Appointment → Treatment
- Many-to-One → Doctor → Department
- Many-to-One → DoctorAvailability → Doctor
- Many-to-One → Appointment → Doctor
- Many-to-One → Appointment → Patient
- One-to-One → Appointment → Payment
- Many-to-One → Payment → Appointment
- Many-to-One → Payment → Patient
- Many-to-One → Payment → Doctor



6. API Resource Endpoints

Endpoint	Method	Description
/api/auth/login	POST	Authenticate user and return JWT token
/api/auth/register	POST	Register new patient account
/api/auth/me	GET	Get current authenticated user profile
/api/admin/dashboard	GET	Get total doctors, patients and appointment stats
/api/admin/doctors	GET	List all doctors with optional search
/api/admin/doctors	POST	Add new doctor profile
/api/admin/doctors/<id>	PUT / DELETE	Update or deactivate a doctor
/api/admin/patients	GET	List all patients, search by name/phone/email/ID
/api/admin/appointments	GET	List all appointments with optional status filter
/api/admin/departments	GET / POST	List or add departments
/api/doctor/dashboard	GET	Get upcoming appointments and patient count
/api/doctor/appointments/<id>/treatment	POST	Add diagnosis, prescription and notes
/api/doctor/patients/<id>/history	GET	Full treatment history for a patient
/api/doctor/availability	GET / POST	View or set availability slots
/api/patient/doctors	GET	Search doctors by name and department
/api/patient/appointments	GET / POST	View or book appointments
/api/patient/appointments/<id>/reschedule	PUT	Reschedule a booked appointment

/api/patient/appointments/<id>/cancel	PUT	Cancel a booked appointment
/api/patient/export-csv	POST	Trigger async Celery job to email treatment history CSV

YAML API Definition File:
Included separately in the submission ZIP as `api.yaml`.

---

7. Architecture and Features

Architecture Overview:

- **/backend/app.py** – main Flask application entry point
- **/backend/models** – SQLAlchemy ORM models using SQLAlchemy
- **/backend/routes** – Flask Blueprints for user and activity routes
- **/backend/celery_tasks.py** – Celery background async tasks like exports and reminders
- **/frontend/src/components** – Vue components for the dynamic UI

Implemented Features:

- Role-based access control for Admin, Doctor, and Patient using JWT authentication
- Admin dashboard with appointment statistics visualised using Chart.js doughnut chart
- Doctor availability management for next 7 days with time-window enforcement on booking
- Appointment booking, rescheduling, and cancellation by patients
- Conflict prevention — duplicate appointments at same date/time for same doctor are blocked
- Treatment records with diagnosis, prescription, notes, and next visit date added by doctors
- Redis caching with SimpleCache fallback
- Responsive UI built with Bootstrap 5 and custom CSS
- Backend Celery async jobs for email reminders, csv exports and monthly reports.

Additional Features:

- ChartsJS for visualization and analysis features using graphs and charts
- Unified responsive UI that works across all screen sizes (Smartphones and Desktops)
- Payment portal with revenue analysis dashboard as admin login.

---

8. Video Presentation

Drive Link: